Chapter 2 worked solutions

Recurrence and Encoder–Decoder Models

These written solutions cover Exercises 2–8 and 12. The companion notebook covers Exercises 1 and 9–11, which ask you to check the arithmetic against code or to run the color-reversal model. The first problems follow single numbers through recurrent updates. The later problems track the same ideas through a batch and across the connection from encoder to decoder.

2. Reverse the order

Exercise 1 read the inputs in the order $0.2, 0.4, -0.1$. Now read the same three inputs in the order $-0.1, 0.4, 0.2$. Start again at zero. Each step multiplies the previous state by 0.5, adds the current input, and applies $\tanh$. Call the number immediately before $\tanh$ the **pre-activation** a_i. The updated state is $h_i = \tanh(a_i)$.

At the first position, the state contribution is $0.5(0) = 0$, so $a_1 = -0.1$. The result $h_1 \approx -0.099667995$ becomes the previous state at position 2. Its negative contribution reduces the input 0.4 to a pre-activation of about 0.350166003. The resulting state $h_2 \approx 0.336522756$ is positive, so at the last position it adds about 0.168261378 to the input 0.2.

Position i	Input x_i	$a_i = 0.5h_{i-1} + x_i$	h_i to six decimals
1	−0.1	$0.5(0) - 0.1 = -0.1$	−0.099668
2	0.4	$0.5(-0.099667995\ldots) + 0.4 \approx 0.350166003$	0.336523
3	0.2	$0.5(0.336522756\ldots) + 0.2 \approx 0.368261378$	0.352470

The states in the last column are rounded for display. Each row uses the previous state's full calculator value. The reversed order gives $h_3 \approx 0.352470$, compared with 0.129806 in Exercise 1. The original final input is −0.1, added to $0.5h_2 \approx 0.230542222$. The reversed final input is 0.2, added to $0.5h_2 \approx 0.168261378$. Both terms in the last update differ.

Even the middle input, 0.4, gives different results because the two runs reach it with different previous states. The parameters are shared across positions, so the same coefficients and nonlinearity are used every time. The argument supplied to that rule changes. An input contributes directly when it arrives and can influence later steps only through the state it helped create. Changing the order therefore changes how the inputs pass through the chain of updates.

3. Compare two gradient paths

Think of nudging an early state and asking how much that nudge changes a state 20 updates later. A **local derivative** gives the sensitivity across one update. The chain rule multiplies the local sensitivities along the path. For example, two successive sensitivities of 0.5 leave $0.5 \times 0.5 = 0.25$ of the original effect. Twenty factors leave far less.

Path factor per step	Product over 20 steps	Approximate decimal
0.5	$0.5^{20} = 1/1,048,576$	0.000000953674
0.99	0.99^{20}	0.817906938

For the chapter's scalar RNN, the local derivative is $0.5(1 - h_i^2)$. The factor 0.5 comes from the recurrent weight and $1 - h_i^2$ comes from differentiating $\tanh$. Because $|h_i|$ is at most one, the magnitude of each factor is at most 0.5. Thus $0.5^{20} \approx 9.54 \times 10^{-7}$ is an **upper bound** on the magnitude of the derivative through that 20-update state chain. The actual value can be smaller. The tiny product explains why a loss signal can have very little effect on learning about an early step along such a path.

For one coordinate of an LSTM, the old cell contributes to the new cell through multiplication by the forget gate. Follow only that direct cell-to-cell connection and hold the gate values fixed. If the forget value is 0.99 for all 20 updates, the product of the direct derivatives is 0.99^{20}. About 81.8% of a small change survives along that path. The cell adds the input-gated candidate at each step, so the calculation describes the retained contribution of a perturbation to the old cell.

In a full LSTM, the gate values may vary across coordinates, examples, and positions. Even along the direct path, a small forget value can make the product small. There are also indirect routes. A change to the cell can alter the hidden state, and that hidden state helps compute later gates and candidates. The prediction depends on the output gate, the transformation of the cell through $\tanh$, and the output layer. The gradient reaching a parameter therefore depends on these routes and on the loss. The value 0.817906938 isolates one route under specified gate values.

4. Open and close a gate

The old cell holds 0.5. The candidate is -0.4, and the original input gate admits half of it. Think of the update as adding two signed contributions. The forget gate controls the retained old value and the input gate controls the new write. After adding those contributions, apply $\tanh$ and multiply by the unchanged output gate 0.6.

$$c_i = f_i(0.5) + i_i(-0.4), \qquad h_i = 0.6 \tanh(c_i).$$

The chapter starts with $f_i = 0.8$ and $i_i = 0.5$. This retains $0.8(0.5) = 0.4$, writes $0.5(-0.4) = -0.2$, and yields $c_i = 0.2$ and $h_i \approx 0.118425$. Each new case below changes only the indicated gate relative to that baseline.

Case	Old contribution	Candidate contribution	c_i	h_i
Forget gate closed, $f_i = 0$	0	-0.2	-0.2	$0.6 \tanh(-0.2) \approx -0.118425$
Forget gate open, $f_i = 1$	0.5	-0.2	0.3	$0.6 \tanh(0.3) \approx 0.174788$
Input gate closed, $i_i = 0$	0.4	0	0.4	$0.6 \tanh(0.4) \approx 0.227969$

With $f_i = 0$, the old cell contributes zero. The negative write still enters, so both the resulting cell and the exposed hidden coordinate are negative. With $f_i = 1$, the entire old value enters the sum, but the negative write still reduces the new cell from 0.5 to 0.3. An open forget gate preserves the old **contribution** to the update.

For the last case, restore f_i to 0.8 before setting $i_i = 0$. The cell keeps 0.4 and receives no write. This gives the largest c_i of the three cases because the original candidate would have subtracted 0.2. A positive candidate could make the comparison go the other way. In all three cases the output gate acts only after the sum, so h_i and c_i are different quantities. A sigmoid at a finite input gives a gate value strictly between zero and one. Here the endpoints make the effect of each gate easy to isolate.

5. Pass the right state

Imagine encoding five source sequences together. Each sequence produces a final hidden vector of width 64 and a final cell vector of width 64. With one layer and one direction, each returned PyTorch state tensor has shape

$$(1, 5, 64) = (\text{layer and direction},\ \text{batch item},\ \text{state coordinate}).$$

The first axis has size one because this encoder has one layer and reads forward only. The middle axis identifies the five source examples. The last axis holds the 64 numerical coordinates of one state. There is no source-position axis in either **final** state tensor. If the examples have different true lengths, each index along the batch axis selects the state at that example's last real source token. Packing the inputs with their true lengths gives that behavior in the chapter's implementation.

For a source with n real tokens, the decoder's initial hidden state receives the encoder's final hidden state $\mathbf{h}_n^{enc}$. Its initial cell state receives the encoder's final cell state $\mathbf{c}_n^{enc}$. Both transfers are direct because the two LSTMs have matching state shapes. At the first decoder update, the hidden state helps compute the decoder's gates and candidate together with the `BOS` embedding. The initial cell supplies the value that the forget gate scales.

An embedding lookup supplies one vector associated with one token ID. For example, two sequences that both end in *red* retrieve the same source embedding for that last token. Their final encoder states can differ because the update for *red* also receives the hidden and cell states produced by their earlier tokens. Passing the last embedding alone would discard that recurrent computation and provide neither state pair nor preceding context. The source embedding in this model has width 16, so it also differs in shape from either of the two width-64 state vectors.

6. Shift before grading

Start with the source *red blue green yellow*. Reverse the four colors to obtain *yellow green blue red*. The model also needs to learn to stop, so append `EOS` to the **labels**. To form the decoder inputs, put `BOS` first and then supply each reference color one step after its own prediction.

Output step	Current decoder input	Reference label
1	`BOS`	*yellow*
2	*yellow*	*green*
3	*green*	*blue*
4	*blue*	*red*
5	*red*	`EOS`

The decoder receives **green** at the step graded against **blue**. By then its recurrent state also reflects the earlier decoder inputs and the initial state pair from the encoder. Under teacher forcing, green is supplied from the reference even if the model's most likely prediction at step 2 was some other color. Blue is supplied at step 4, after it has served as a label at step 3.

There are five inputs and five labels. The two lists align because `BOS` starts the input and `EOS` ends the labels. The source has four color positions. The extra decoder step learns to end the output, and the reversal acts on the colors before the boundary tokens are added.

7. Count prediction positions

Consider batching the three-color source *red blue green* with the five-color source *yellow green red blue black*. The shorter source needs two right-padding positions to share a rectangular array with the longer one. Their true lengths remain three and five. The vocabulary has nine entries, including three special tokens, and the source embedding has width 16.

Each target reverses the real source colors and appends `EOS`, giving four labels in the first row and six in the second. Pad the first target row to six positions as well.

Row	Source IDs shown as tokens	Target labels shown as tokens
1	red blue green PAD PAD	green blue red EOS PAD PAD
2	yellow green red blue black	black blue red green yellow EOS

Quantity	Axes	Shape
Source IDs	batch, padded source position	$(2, 5)$
Source embeddings	batch, padded source position, embedding coordinate	$(2, 5, 16)$
Target labels	batch, padded prediction position	$(2, 6)$
Logits	batch, padded prediction position, vocabulary class	$(2, 6, 9)$

For the source, lookup appends an embedding axis to the rectangular ID array. The encoder then uses true lengths to process only real source positions. For the target, the shifted decoder input also has shape $(2, 6)$. Its six columns lead the output projection to produce six vocabulary-score rows per example. The last axis of the logits has width nine because each row has a score for every vocabulary class. In contrast, each label position stores just one reference ID.

The rectangular target has $2 \times 6 = 12$ slots. The loss counts $4 + 6 = 10$ valid prediction positions. That is eight color labels and two endings. The two `PAD` labels in the shorter row are excluded. With the chapter's token-mean loss, we add ten negative log-probabilities and divide by ten. Counting the rectangular width as six for both examples would incorrectly include two storage positions. The exclusion applies to positions labeled `PAD`. The `PAD` class is still one of the nine alternatives in the distribution at each valid position.

8. Recalculate the loss

The chapter's reference continuation is *el coche rojo* followed by `EOS`. At each step we look up the probability of the correct token under the given source and the correct preceding target tokens. A smaller probability incurs a larger penalty. The penalty for one step is the negative natural logarithm, $-\ln p$.

Reference label	Original p	Original $-\ln p$	p with weaker ending	New $-\ln p$
el	0.80	0.223144	0.80	0.223144
coche	0.60	0.510826	0.60	0.510826
rojo	0.75	0.287682	0.75	0.287682
`EOS`	0.90	0.105361	0.10	2.302585

In the original case, the four conditional probabilities multiply to $0.8 \times 0.6 \times 0.75 \times 0.9 = 0.324$. Adding logs is equivalent to taking the log of the product. Using unrounded values, the sum of the four penalties is about 1.127011763. The mean is

$$\ell_{original} = -\frac{\ln(0.324)}{4} \approx 0.281753.$$

After lowering only the correct ending's probability, the product is $0.8 \times 0.6 \times 0.75 \times 0.1 = 0.036$. The sum of the penalties is about 3.324236341, so the mean is

$$\ell_{new} = -\frac{\ln(0.036)}{4} \approx 0.831059.$$

The mean increases by about 0.549306 natural-log units per token. A shortcut is to subtract just the ending penalties, since the other three steps cancel. The change in the sum is $-\ln(0.1) + \ln(0.9) = \ln 9$. The denominator stays four, so the change in the mean is $(\ln 9)/4$.

After the three correct words, the reference still specifies what should happen next. The decoder should stop. Giving less probability to `EOS` puts more probability on other continuations at that step and reduces the probability of completing the exact reference with its ending. This is why the ending contributes to the score alongside the ordinary target tokens. The product is the probability assigned to the reference path under these conditional distributions. Greedy generation instead selects the largest-probability option at every step.

12. Locate the information path

The encoder reads *the*, *red*, and *car* in that order. When it reads *car*, its third update combines that token's source embedding with the preceding hidden and cell states. The new pair can carry information about *car* through both states. The decoder receives this pair before it makes any target predictions.

Let $n = 3$ be the final **source** position. Let t count **target** prediction steps, beginning at $t = 1$ with `BOS` as input. The state labels in the diagram use those separate counters.

$$\begin{aligned}
\text{source embedding of car} &\rightarrow (\mathbf{h}_3^{enc}, \mathbf{c}_3^{enc}) \\
&\rightarrow (\mathbf{h}_0^{dec}, \mathbf{c}_0^{dec}) \\
&\xrightarrow[\text{input BOS}]{} (\mathbf{h}_1^{dec}, \mathbf{c}_1^{dec}) \\
&\xrightarrow[\text{input el}]{} (\mathbf{h}_2^{dec}, \mathbf{c}_2^{dec}) \\
&\rightarrow \text{logits for target step 2} \rightarrow p_2(\text{coche}).
\end{aligned}$$

At step 1 the decoder updates the transferred pair while receiving `BOS`. At step 2 it receives *el* under teacher forcing, or as feedback if it generated *el* at step 1. Both decoder states from step 1 feed into step 2. The final projection for this prediction reads $\mathbf{h}_2^{dec}$. Softmax converts the logits to the distribution p_2 over the target vocabulary. The cell can affect that output by influencing the updated hidden state. The source word *car* can therefore affect the score for *coche* through the encoder's final state pair and the decoder's recurrent updates. How much of that information survives depends on the learned updates.

To give the decoder additional access, keep a source representation for every real source position. At each target step, let the decoder consult those representations while forming its prediction. For *coche*, it could use the representation associated with *car*. Later, for *rojo*, it could use the one associated with *red*. The useful change is that the source information available at a target step can depend on which word the decoder is currently trying to predict.